

Tutorial 3 – Create a Jigsaw Board with Image

Create a new scene and call it JigsawBoard as this is a scene to test out the various features.

Next, create a script named BoardGen.

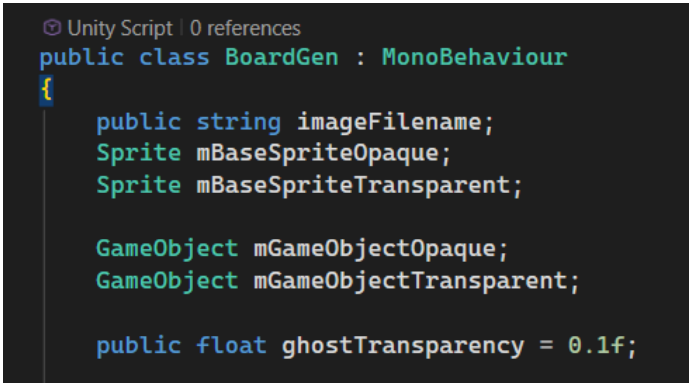
Member Variable

Let's add a variable public string imageFilename as this variable will hold the filename of the image that will be used to generate the Jigsaw board.

Then, add a mGameObjectOpaque and mGameTransparent of type Sprite. The first variable stores the base opaque sprite loaded from the image file.

Next, we will create two game objects named mGameObjectOpaque and mGameTransparent as it'll display the opaque sprite and the GameObject and will display the transparent sprite.

Finally, we'll add a variable called ghostTransparency to hold the transparency level of the ghost pixel in the game board.

A screenshot of a Unity Script editor showing the code for a class named BoardGen. The script is a C# MonoBehaviour. It contains several public member variables: a string imageFilename, two Sprite variables mBaseSpriteOpaque and mBaseSpriteTransparent, two GameObject variables mGameObjectOpaque and mGameObjectTransparent, and a float variable ghostTransparency initialized to 0.1f. The code is color-coded with blue for keywords, green for types, and black for variable names and values.

```
Unity Script | 0 references
public class BoardGen : MonoBehaviour
{
    public string imageFilename;
    Sprite mBaseSpriteOpaque;
    Sprite mBaseSpriteTransparent;

    GameObject mGameObjectOpaque;
    GameObject mGameObjectTransparent;

    public float ghostTransparency = 0.1f;
}
```

Load Base Texture

Now, we will create a function called LoadBaseTexture as this function will load the generating a Tile object at the specified position i, j in the jigsaw puzzle grid, based on the provided baseTexture.

```

Sprite LoadBaseTexture()
{
    Texture2D tex = SpriteUtils.LoadTexture(imageFilename);
    if (!tex.isReadable)
    {
        Debug.Log("Error: Texture is not readable");
        return null;
    }

    if (tex.width % Tile.tileSize != 0 || tex.height % Tile.tileSize != 0)
    {
        Debug.Log("Error: Image must be of size that is multiple of <" + Tile.tileSize + ">");
        return null;
    }

    // Add padding to the image.
    Texture2D newTex = new Texture2D(
        tex.width + Tile.padding * 2,
        tex.height + Tile.padding * 2,
        TextureFormat.ARGB32,
        false);

```

```

// Set the default colour as white
for (int x = 0; x < newTex.width; ++x)
{
    for (int y = 0; y < newTex.height; ++y)
    {
        newTex.SetPixel(x, y, Color.white);
    }
}

// Copy the colours.
for (int x = 0; x < tex.width; ++x)
{
    for (int y = 0; y < tex.height; ++y)
    {
        Color color = tex.GetPixel(x, y);
        color.a = 1.0f;
        newTex.SetPixel(x + Tile.padding, y + Tile.padding, color);
    }
}
newTex.Apply();

Sprite sprite = SpriteUtils.CreateSpriteFromTexture2D(
    newTex,
    0,
    0,
    newTex.width,
    newTex.height);
return sprite;

```

It first loads the texture from the image file using the `SpriteUtils.LoadTexture` with the given `imageFilename`. If the texture is not readable, it logs an error message and returns `null`.

It checks if the width and height of the loaded texture are multiples of `Tile.tileSize`. If not, it logs an error message and returns `null`. It then creates a new texture with dimensions increased by twice the padding. The default colour of all pixels in the new texture is set to white.

Afterwards, it'll repeat through each pixel of the original texture and copies the color to correlate with the position in the new texture. The alpha value of each pixel is set to 1.0f.

After, copying the colours, it applies the changes to the new texture using the Apply method.

Start Method

Now, we will implement the start method as it'll start with the function called LoadBaseMethod to load the base opaque sprite form the image file.

Then, create a new empty GameObject and assign it to mGameObjectOpaque to hold the opaque sprite. Next, add a SpriteRenderer component to this game object and set its sprite to the loaded opaque sprite. Then we set the sorting layer for the opaque sprite.

Afterwards, we create a transparent view of the base sprite by calling the CreateTransparentView function. Then add the SpriteRenderer component to mGameObjectTransparent and set its sprite to the transparent sprite.

Next, add a SpriteRenderer component to mGameObjectTransparent set its sprite to the transparent sprite. We also set the sorting layer for the transparent sprite and hide the opaque GameObject by default

Finally, we will set the position of the main camera based on the dimensions of the opaque to ensure a proper view.

```

// Start is called before the first frame update
@ Unity Message | 0 references
void Start()
{
    mBaseSpriteOpaque = LoadBaseTexture();
    mGameObjectOpaque = new GameObject();
    mGameObjectOpaque.name = imageFilename + "_Opaque";
    mGameObjectOpaque.AddComponent<SpriteRenderer>().sprite = mBaseSpriteOpaque;
    mGameObjectOpaque.GetComponent<SpriteRenderer>().sortingLayerName = "Opaque";

    mBaseSpriteTransparent = CreateTransparentView(mBaseSpriteOpaque.texture);
    mGameObjectTransparent = new GameObject();
    mGameObjectTransparent.name = imageFilename + "_Transparent";
    mGameObjectTransparent.AddComponent<SpriteRenderer>().sprite = mBaseSpriteTransparent;
    mGameObjectTransparent.GetComponent<SpriteRenderer>().sortingLayerName = "Transparent";

    mGameObjectOpaque.gameObject.SetActive(false);

    SetCameraPosition();

    // Create the Jigsaw tiles.
    //CreateJigsawTiles();
    StartCoroutine(Coroutine_CreateJigsawTiles());
}

```

Create Transparent View

Now, we will implement the CreateTransparentView that we used in the Start method as it'll be responsible for creating a transparent view of the provided texture.

```

Sprite CreateTransparentView(Texture2D tex)
{
    Texture2D newTex = new Texture2D(
        tex.width,
        tex.height,
        TextureFormat.ARGB32,
        false);

    for (int x = 0; x < newTex.width; x++)
    {
        for (int y = 0; y < newTex.height; y++)
        {
            Color c = tex.GetPixel(x, y);
            if (x > Tile.padding &&
                x < (newTex.width - Tile.padding) &&
                y > Tile.padding &&
                y < (newTex.height - Tile.padding))
            {
                c.a = ghostTransparency;
            }
            newTex.SetPixel(x, y, c);
        }
    }
    newTex.Apply();

    Sprite sprite = SpriteUtils.CreateSpriteFromTexture2D(
        newTex,
        0,
        0,
        newTex.width,
        newTex.height);
    return sprite;
}

```

Once the

Set Camera Position

Now, we will implement the `SetMethodPosition` which adjusts the position size of the main camera based on the dimensions of the opaque sprite. It'll position the camera at the center of the sprite's texture. This method ensures that the camera provides a suitable view of the board.

```
void SetCameraPosition()
{
    Camera.main.transform.position = new Vector3(mBaseSpriteOpaque.texture.width / 2,
        mBaseSpriteOpaque.texture.height / 2, -10.0f);
    Camera.main.orthographicSize = mBaseSpriteOpaque.texture.width / 2;
}
```

Create Jigsaw Tile

Now, we will add some more member ranges. Go back to `BoardGen` and continue adding the member variables integer `numTilesX` and `numTilesY`. These are public properties representing the number of tiles in the horizontal and vertical axes of the jigsaw puzzle, respectively.

We then add a two-dimensional array of `Tile` objects representing the individual pieces of the jigsaw puzzle.

Lastly, we will add a transform variable called `parentForTiles` as this represents the parent transform under which the jigsaw puzzle tiles will be instantiated as child objects.

```

public class BoardGen : MonoBehaviour
{
    public string imageFilename;
    Sprite mBaseSpriteOpaque;
    Sprite mBaseSpriteTransparent;

    GameObject mGameObjectOpaque;
    GameObject mGameObjectTransparent;

    public float ghostTransparency = 0.1f;

    // Jigsaw tiles creation.
    9 references
    public int numTileX { get; private set; }
    9 references
    public int numTileY { get; private set; }

    Tile[,] mTiles = null;
    GameObject[,] mTileGameObjects = null;

    public Transform parentForTiles = null;

```

Create Game Object from Tile

Now we will implement the methods. Firstly, we will add CreateGameObjectFromTile.

```

public static GameObject CreateGameObjectFromTile(Tile tile)
{
    GameObject obj = new GameObject();

    obj.name = "TileGameObe_" + tile.xIndex.ToString() + "_" + tile.yIndex.ToString();

    obj.transform.position = new Vector3(tile.xIndex * Tile.tileSize, tile.yIndex * Tile.tileSize, 0.0f);

    SpriteRenderer spriteRenderer = obj.AddComponent<SpriteRenderer>();
    spriteRenderer.sprite = SpriteUtils.CreateSpriteFromTexture2D(
        tile.finalCut,
        0,
        0,
        Tile.padding * 2 + Tile.tileSize,
        Tile.padding * 2 + Tile.tileSize);

    BoxCollider2D box = obj.AddComponent<BoxCollider2D>();
    return obj;
}

```

We begin by creating a new GameObject instance and assign it a name indicating its position within the grid. The position of the GameObject is then set using the xIndex and yIndex properties of the Tile, considering the tileSize. A SpriteRenderer component is added to the GameObject to render the visual representation of the tile, utilising a sprite created from the **finalCut** texture of the tile.

Additionally, we add a BoxCollider2D component to facilitate the picking and selection of the tiles. Finally, the function returns the generated GameObject.

Create Jigsaw Tiles

Next, we will create the CreateJigsawTiles function as it generates the jigsaw puzzle tiles based on the opaque base texture.

```
void CreateJigsawTiles()
{
    Texture2D baseTexture = mBaseSpriteOpaque.texture;
    numTileX = baseTexture.width / Tile.tileSize;
    numTileY = baseTexture.height / Tile.tileSize;

    mTiles = new Tile[numTileX, numTileY];
    mTileGameObjects = new GameObject[numTileX, numTileY];

    for (int i = 0; i < numTileX; i++)
    {
        for (int j = 0; j < numTileY; j++)
        {
            mTiles[i, j] = CreateTile(i, j, baseTexture);
            mTileGameObjects[i, j] = CreateGameObjectFromTile(mTiles[i, j]);
            if (parentForTiles != null)
            {
                mTileGameObjects[i, j].transform.SetParent(parentForTiles);
            }
        }
    }
}
```

Then, we calculate the number of tiles in both the horizontal and vertical axes by dividing the width and height of the base texture by the tileSize.

Next, we will calculate the array mTiles and mTileGameObjects to store Tile objects and corresponding Game Objects instances for each tile position.

Afterwards, we create a game object using the CreateGameObjectFromTile function.

Create Tile

The CreateTile method generates a Tile object at the specified position I, j in the jigsaw puzzle grid, based on the provided baseTexture.

```

Tile CreateTile(int i, int j, Texture2D baseTexture)
{
    Tile tile = new Tile(baseTexture);
    tile.xIndex = i;
    tile.yIndex = j;

    // Left side tiles.
    if (i == 0)
    {
        tile.SetCurveType(Tile.Direction.LEFT, Tile.PosNegType.NONE);
    }
    else
    {
        // We have to create a tile that has LEFT direction opposite curve type.
        Tile leftTile = mTiles[i - 1, j];
        Tile.PosNegType rightOp = leftTile.GetCurveType(Tile.Direction.RIGHT);
        tile.SetCurveType(Tile.Direction.LEFT, rightOp == Tile.PosNegType.NEG ?
            Tile.PosNegType.POS : Tile.PosNegType.NEG);
    }

    // Bottom side tiles
    if (j == 0)
    {
        tile.SetCurveType(Tile.Direction.DOWN, Tile.PosNegType.NONE);
    }
    else
    {
        Tile downTile = mTiles[i, j - 1];
        Tile.PosNegType upOp = downTile.GetCurveType(Tile.Direction.UP);
        tile.SetCurveType(Tile.Direction.DOWN, upOp == Tile.PosNegType.NEG ?
            Tile.PosNegType.POS : Tile.PosNegType.NEG);
    }

    // Right side tiles.
    if (i == numTileX - 1)
    {
        tile.SetCurveType(Tile.Direction.RIGHT, Tile.PosNegType.NONE);
    }
    else
    {
        float toss = Random.Range(0f, 1f);
        if (toss < 0.5f)
        {
            tile.SetCurveType(Tile.Direction.RIGHT, Tile.PosNegType.POS);
        }
        else
        {
            tile.SetCurveType(Tile.Direction.RIGHT, Tile.PosNegType.NEG);
        }
    }
}

```

```

// Up side tile.
if (j == numTileY - 1)
{
    tile.SetCurveType(Tile.Direction.UP, Tile.PosNegType.NONE);
}
else
{
    float toss = Random.Range(0f, 1f);
    if (toss < 0.5f)
    {
        tile.SetCurveType(Tile.Direction.UP, Tile.PosNegType.POS);
    }
    else
    {
        tile.SetCurveType(Tile.Direction.UP, Tile.PosNegType.NEG);
    }
}

tile.Apply();
return tile;

```

Now, we have created a new Tile object using the provided baseTexture as a parameter. We set the xIndex and yIndex properties of the tile to the values i and j, respectively, indicating its position within the grid. For left-side tiles, if the tile is at the leftmost edge (i equal to 0), we set its left curve type to NONE.

Otherwise, we determine the curve type of the left side based on the curve type of the tile on its left. If the left tile's right curve type is NEG, we set the current tile's left curve type to POS, and vice versa.

For bottom side tiles, if the tile is at the bottom most edge (j equal to 0), we set its bottom curve type to NONE. Otherwise, we determine the curve type of the bottom side based on the curve type of the tile below it.

Finally, we apply the changes made to the tile's curve types using the Apply method as it'll finalize the finalCut image with flood fill.

Now, we will go to the Start method and add the function call to CreateJigsawTiles.

```
// Start is called before the first frame update
@ Unity Message | 0 references
void Start()
{
    mBaseSpriteOpaque = LoadBaseTexture();
    mGameObjectOpaque = new GameObject();
    mGameObjectOpaque.name = imageFilename + "_Opaque";
    mGameObjectOpaque.AddComponent<SpriteRenderer>().sprite = mBaseSpriteOpaque;
    mGameObjectOpaque.GetComponent<SpriteRenderer>().sortingLayerName = "Opaque";

    mBaseSpriteTransparent = CreateTransparentView(mBaseSpriteOpaque.texture);
    mGameObjectTransparent = new GameObject();
    mGameObjectTransparent.name = imageFilename + "_Transparent";
    mGameObjectTransparent.AddComponent<SpriteRenderer>().sprite = mBaseSpriteTransparent;
    mGameObjectTransparent.GetComponent<SpriteRenderer>().sortingLayerName = "Transparent";

    mGameObjectOpaque.gameObject.SetActive(false);

    SetCameraPosition();

    // Create the Jigsaw tiles.
    //CreateJigsawTiles();
    StartCoroutine(Coroutine_CreateJigsawTiles());
}
```

Final Script:

Board

```

using System.Collections;
using System.Collections.Generic;
using UnityEngine;

© Unity Script | 0 references
public class BoardGen : MonoBehaviour
{
    public string imageFilename;
    Sprite mBaseSpriteOpaque;
    Sprite mBaseSpriteTransparent;

    GameObject mGameObjectOpaque;
    GameObject mGameObjectTransparent;

    public float ghostTransparency = 0.1f;

    // Jigsaw tiles creation.
    9 references
    public int numTileX { get; private set; }
    9 references
    public int numTileY { get; private set; }

    Tile[,] mTiles = null;
    GameObject[,] mTileGameObjects = null;

    public Transform parentForTiles = null;

    1 reference
    Sprite LoadBaseTexture()
    {
        Texture2D tex = SpriteUtils.LoadTexture(imageFilename);
        if (!tex.isReadable)
        {
            Debug.Log("Error: Texture is not readable");
            return null;
        }

        if (tex.width % Tile.tileSize != 0 || tex.height % Tile.tileSize != 0)
        {
            Debug.Log("Error: Image must be of size that is multiple of <" + Tile.tileSize + ">");
            return null;
        }

        // Add padding to the image.
        Texture2D newTex = new Texture2D(
            tex.width + Tile.padding * 2,
            tex.height + Tile.padding * 2,
            TextureFormat.ARGB32,
            false);
    }

```

```

    // Set the default colour as white
    for (int x = 0; x < newTex.width; ++x)
    {
        for (int y = 0; y < newTex.height; ++y)
        {
            newTex.SetPixel(x, y, Color.white);
        }
    }

    // Copy the colours.
    for (int x = 0; x < tex.width; ++x)
    {
        for (int y = 0; y < tex.height; ++y)
        {
            Color color = tex.GetPixel(x, y);
            color.a = 1.0f;
            newTex.SetPixel(x + Tile.padding, y + Tile.padding, color);
        }
    }
    newTex.Apply();

    Sprite sprite = SpriteUtils.CreateSpriteFromTexture2D(
        newTex,
        0,
        0,
        newTex.width,
        newTex.height);
    return sprite;

```

```

// Start is called before the first frame update
@ Unity Message | 0 references
void Start()
{
    mBaseSpriteOpaque = LoadBaseTexture();
    mGameObjectOpaque = new GameObject();
    mGameObjectOpaque.name = imageFilename + "_Opaque";
    mGameObjectOpaque.AddComponent<SpriteRenderer>().sprite = mBaseSpriteOpaque;
    mGameObjectOpaque.GetComponent<SpriteRenderer>().sortingLayerName = "Opaque";

    mBaseSpriteTransparent = CreateTransparentView(mBaseSpriteOpaque.texture);
    mGameObjectTransparent = new GameObject();
    mGameObjectTransparent.name = imageFilename + "_Transparent";
    mGameObjectTransparent.AddComponent<SpriteRenderer>().sprite = mBaseSpriteTransparent;
    mGameObjectTransparent.GetComponent<SpriteRenderer>().sortingLayerName = "Transparent";

    mGameObjectOpaque.gameObject.SetActive(false);

    SetCameraPosition();

    // Create the Jigsaw tiles.
    //CreateJigsawTiles();
    StartCoroutine(Coroutine_CreateJigsawTiles());
}

```

```

Sprite CreateTransparentView(Texture2D tex)
{
    Texture2D newTex = new Texture2D(
        tex.width,
        tex.height,
        TextureFormat.ARGB32,
        false);

    for (int x = 0; x < newTex.width; x++)
    {
        for (int y = 0; y < newTex.height; y++)
        {
            Color c = tex.GetPixel(x, y);
            if (x > Tile.padding &&
                x < (newTex.width - Tile.padding) &&
                y > Tile.padding &&
                y < (newTex.height - Tile.padding))
            {
                c.a = ghostTransparency;
            }
            newTex.SetPixel(x, y, c);
        }
    }
    newTex.Apply();

    Sprite sprite = SpriteUtils.CreateSpriteFromTexture2D(
        newTex,
        0,
        0,
        newTex.width,
        newTex.height);
    return sprite;
}

```

```

void SetCameraPosition()
{
    Camera.main.transform.position = new Vector3(mBaseSpriteOpaque.texture.width / 2,
        mBaseSpriteOpaque.texture.height / 2, -10.0f);
    Camera.main.orthographicSize = mBaseSpriteOpaque.texture.width / 2;
}

2 references
public static GameObject CreateGameObjectFromTile(Tile tile)
{
    GameObject obj = new GameObject();

    obj.name = "TileGameObe_" + tile.xIndex.ToString() + "_" + tile.yIndex.ToString();

    obj.transform.position = new Vector3(tile.xIndex * Tile.tileSize, tile.yIndex * Tile.tileSize, 0.0f);

    SpriteRenderer spriteRenderer = obj.AddComponent<SpriteRenderer>();
    spriteRenderer.sprite = SpriteUtils.CreateSpriteFromTexture2D(
        tile.finalCut,
        0,
        0,
        Tile.padding * 2 + Tile.tileSize,
        Tile.padding * 2 + Tile.tileSize);

    BoxCollider2D box = obj.AddComponent<BoxCollider2D>();
    return obj;
}

0 references
void CreateJigsawTiles()
{
    Texture2D baseTexture = mBaseSpriteOpaque.texture;
    numTileX = baseTexture.width / Tile.tileSize;
    numTileY = baseTexture.height / Tile.tileSize;

    mTiles = new Tile[numTileX, numTileY];
    mTileGameObjects = new GameObject[numTileX, numTileY];

    for (int i = 0; i < numTileX; i++)
    {
        for (int j = 0; j < numTileY; j++)
        {
            mTiles[i, j] = CreateTile(i, j, baseTexture);
            mTileGameObjects[i, j] = CreateGameObjectFromTile(mTiles[i, j]);
            if (parentForTiles != null)
            {
                mTileGameObjects[i, j].transform.SetParent(parentForTiles);
            }
        }
    }
}

```

```

Tile CreateTile(int i, int j, Texture2D baseTexture)
{
    Tile tile = new Tile(baseTexture);
    tile.xIndex = i;
    tile.yIndex = j;

    // Left side tiles.
    if (i == 0)
    {
        tile.SetCurveType(Tile.Direction.LEFT, Tile.PosNegType.NONE);
    }
    else
    {
        // We have to create a tile that has LEFT direction opposite curve type.
        Tile leftTile = mTiles[i - 1, j];
        Tile.PosNegType rightOp = leftTile.GetCurveType(Tile.Direction.RIGHT);
        tile.SetCurveType(Tile.Direction.LEFT, rightOp == Tile.PosNegType.NEG ?
            Tile.PosNegType.POS : Tile.PosNegType.NEG);
    }

    // Bottom side tiles
    if (j == 0)
    {
        tile.SetCurveType(Tile.Direction.DOWN, Tile.PosNegType.NONE);
    }
    else
    {
        Tile downTile = mTiles[i, j - 1];
        Tile.PosNegType upOp = downTile.GetCurveType(Tile.Direction.UP);
        tile.SetCurveType(Tile.Direction.DOWN, upOp == Tile.PosNegType.NEG ?
            Tile.PosNegType.POS : Tile.PosNegType.NEG);
    }

    // Right side tiles.
    if (i == numTileX - 1)
    {
        tile.SetCurveType(Tile.Direction.RIGHT, Tile.PosNegType.NONE);
    }
    else
    {
        float toss = Random.Range(0f, 1f);
        if (toss < 0.5f)
        {
            tile.SetCurveType(Tile.Direction.RIGHT, Tile.PosNegType.POS);
        }
        else
        {
            tile.SetCurveType(Tile.Direction.RIGHT, Tile.PosNegType.NEG);
        }
    }
}

```

```

// Up side tile.
if (j == numTileY - 1)
{
    tile.SetCurveType(Tile.Direction.UP, Tile.PosNegType.NONE);
}
else
{
    float toss = Random.Range(0f, 1f);
    if (toss < 0.5f)
    {
        tile.SetCurveType(Tile.Direction.UP, Tile.PosNegType.POS);
    }
    else
    {
        tile.SetCurveType(Tile.Direction.UP, Tile.PosNegType.NEG);
    }
}

tile.Apply();
return tile;

```